Endpoint Attack Detection & Investigation Lab

Microsoft Sentinel + Sysmon + Windows Endpoint

Project Goal: Build an endpoint detection lab that demonstrates host telemetry collection, SIEM ingestion, threat hunting, and alert engineering using Microsoft Sentinel.

---

1) Executive Summary

This project simulates a modern SOC workflow by forwarding **Sysmon endpoint telemetry** from a Windows virtual machine into **Microsoft Sentinel** through a **Log Analytics Workspace**. The lab focuses on detecting attacker tradecraft at the endpoint level, beginning with suspicious **Base64-encoded PowerShell execution**, then expanding into persistence and post-exploitation detections.

---

2) Lab Architecture

Components

- **Windows VM:** `JozefsVM`
- **Telemetry Agent:** Azure Monitor Agent
- **Endpoint Logging:** Sysmon
- **SIEM:** Microsoft Sentinel
- **Workspace:** `JB-LAW`
- **Log Source:** `Microsoft-Windows-Sysmon/Operational`

Data Flow

1. Attack activity occurs on the Windows endpoint
2. Sysmon captures process creation events
3. Azure Monitor Agent forwards logs via DCR
4. Logs land in Log Analytics workspace
5. Microsoft Sentinel queries generate detections
6. Alerts are converted into incidents for investigation

---

3) Telemetry Enablement

Sysmon Deployment

Sysmon was deployed directly on **JozefsVM** using the Microsoft Sys-Internals Sysmon64 binary and the **SwiftOnSecurity Sysmon configuration** to provide enterprise-style endpoint telemetry.

Installation steps executed on the VM:

mkdir C:\Tools\Sysmon

cd C:\Tools\Sysmon

Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile Sysmon.zip

Expand-Archive .\Sysmon.zip -DestinationPath C:\Tools\Sysmon -Force

Invoke-WebRequest -Uri https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml -OutFile sysmonconfig.xml

.\Sysmon64.exe -accepteula -i .\sysmonconfig.xmlValidation

After installation, Sysmon service health and event generation were verified by confirming:

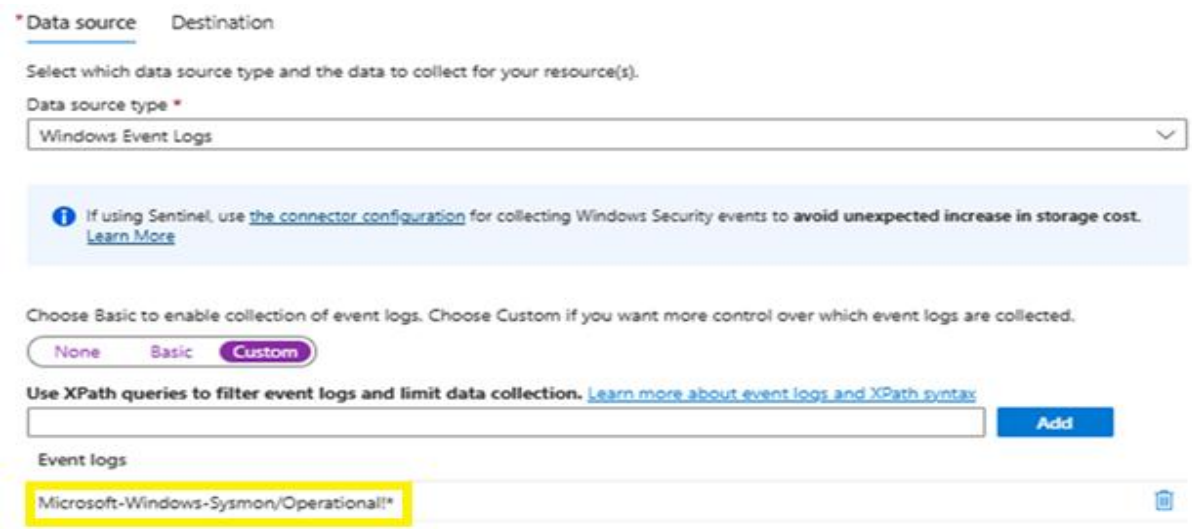
- **Service:** Sysmon64 running
- **Log Channel:** Microsoft-Windows-Sysmon/Operational
- **Primary Event Used:** Event ID 1 (Process Creation)

This telemetry source later became the primary log feed for Sentinel threat hunting and scheduled alert generation.

```
PS C:\Tools\Sysmon> Expand-Archive .\Sysmon.zip -DestinationPath C:\Tools\Sysmon -Force
PS C:\Tools\Sysmon> Invoke-WebRequest -Uri https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmon
config-export.xml -OutFile sysmonconfig.xml
>> .\Sysmon64.exe -accepteula -i .\sysmonconfig.xml


System Monitor v15.20 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.50
Sysmon schema version: 4.91
Configuration file validated.
```

Data Collection Rule (DCR)

A Data Collection Rule was configured to forward Sysmon operational logs from JozefsVM into the JB-LAW Log Analytics workspace.



Telemetry was sourced from Microsoft-Windows-Sysmon/Operational because Sysmon provides enhanced process creation visibility, including full PowerShell command-line logging required for encoded execution detection.

Configuration

- Target VM: JozefsVM
- Data source: Windows Event Logs
- Channel: Microsoft-Windows-Sysmon/Operational
- Destination: JB-LAW
- Agent: Azure Monitor Agent

Purpose

This configuration establishes persistent endpoint log ingestion into **Microsoft Sentinel**, providing the telemetry foundation required for threat hunting, analytic rule development, alert validation, and post-detection investigation.

---

4) Initial Threat Hunt — Encoded PowerShell

Objective

Detect suspicious PowerShell executions using the `-enc` flag, a common obfuscation technique used by attackers.

Hunting Query

```
Event
| where Source == "Microsoft-Windows-Sysmon"
| where EventID == 1
| where RenderedDescription contains "-enc"
```

Detection Logic

This hunt focuses on identifying suspicious PowerShell execution patterns that may indicate attacker activity. The logic is designed to surface:

- Encoded commands using Base64
- Obfuscated or stealthy script execution
- Early-stage execution techniques associated with malicious tradecraft

MITRE ATT&CK Mapping

- **TA0002 — Execution**
- **T1059.001 — PowerShell**

Attack Simulation

To validate the detection logic, an encoded PowerShell command was executed directly on JozefsVM. This emulated adversary tradecraft by leveraging obfuscated PowerShell execution, a technique frequently used by threat actors to bypass basic security controls, obscure payload visibility, and execute malicious code during the initial stages of compromise.

Attack Simulation on Endpoint

Test Command:

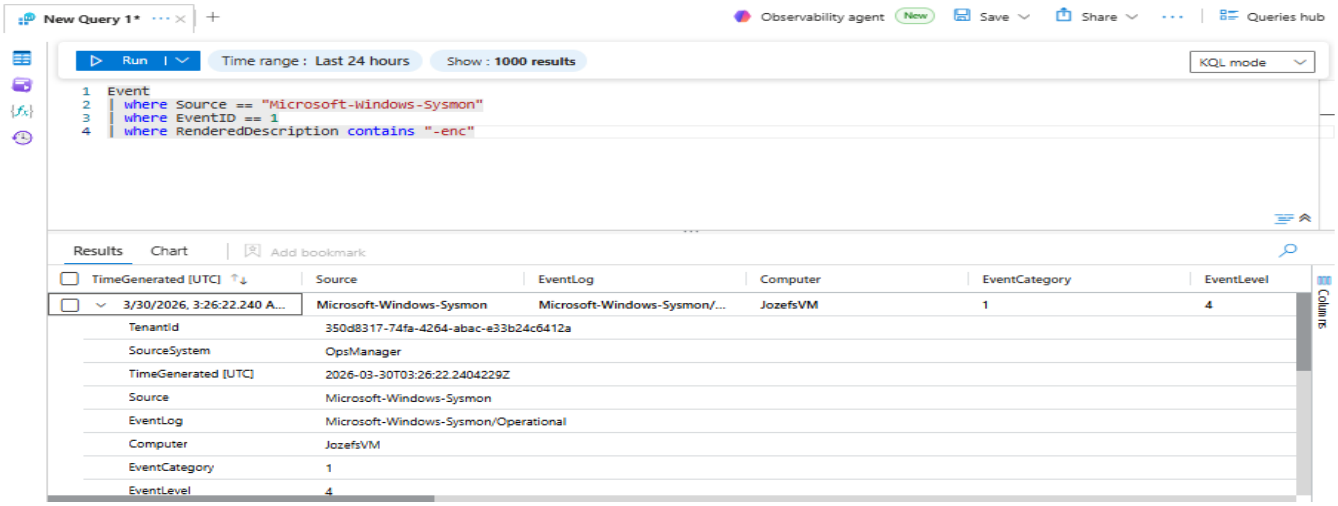
powershell -nop -w hidden -enc SQBmACgAJABQAFMAVgB1AHIAcwBpAG8AbgBUAGEAYgBsAGUALgBQAFMAVgB1AHIAcwBpAG8AbgApAA==

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Jozef> powershell -nop -w hidden -enc SQBmACgAJABQAFMAVgB1AHIAcwBpAG8AbgBUAGEAYgBsAGUALgBQAFMAVgB1AHIAcwBpA
8AbgApAA==
```

Purpose:
This simulated adversary tradecraft using **Base64-encoded PowerShell execution**, a common technique used for:

- malware staging
- defense evasion
- payload execution

Detection Result:
This execution successfully generated a **Sysmon Event ID 1** process creation log and was detected by the Sentinel KQL hunting query.



5) Scheduled Analytics Rule Deployment

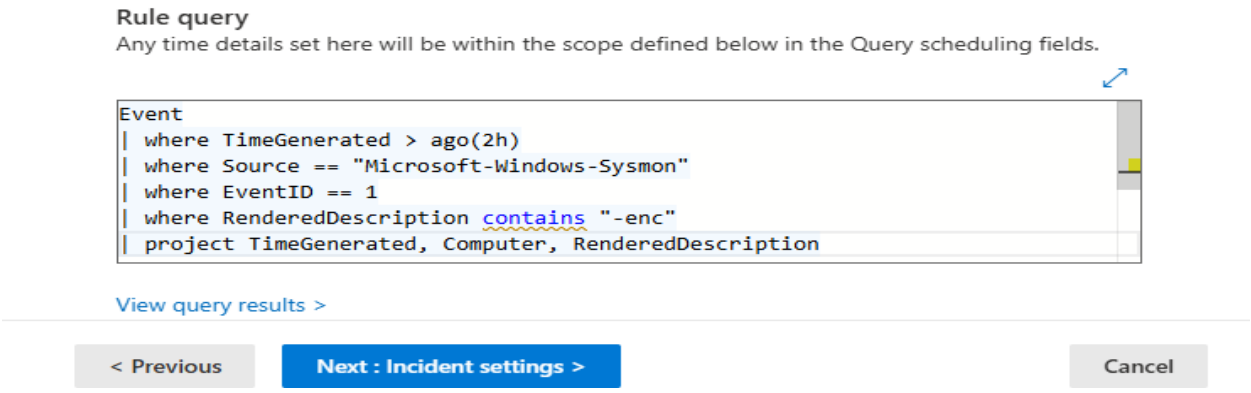
After validating the hunting query, the detection logic was operationalized as a **Scheduled Analytics Rule** in Microsoft Sentinel.

Rule Configuration

- **Rule Name:** POWERSHELL EX
- **Rule Type:** Scheduled query rule
- **Data Source:** Microsoft-Windows-Sysmon/Operational
- **Event ID:** 1
- **Detection Condition:** Command line contains -enc
- **Severity:** High
- **Tactics:** Execution, Defense Evasion
- **Techniques:**
    - T1059.001 — PowerShell

Detection Query

```
Event
| where TimeGenerated > ago(2h)
| where Source == "Microsoft-Windows-Sysmon"
| where EventID == 1
| where RenderedDescription contains "-enc"
| project TimeGenerated, Computer, RenderedDescription
```



Endpoint Validation Script Execution

To validate the scheduled analytics rule, a benign encoded PowerShell script was executed directly on **JozefsVM**.

The purpose of this step was to simulate **obfuscated PowerShell execution behavior** commonly observed during malware staging, payload launch, and post-exploitation scripting.

Validation Script Executed on VM

```
$cmd = 'Write-Output "Sentinel test"'
$enc = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))
powershell.exe -enc $enc
```

Execution Purpose

```
>> $enc = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))
>> powershell.exe -enc $enc
Sentinel test
```

This script safely:

- Generates a benign PowerShell command
- Encodes the command in Base64 Unicode
- Executes the payload using the -enc switch
- Triggers **Sysmon Event ID 1**
- Validates the **Scheduled Analytics Rule**
- Generates a **High-severity alert in the unified Defender incident queue**

Expected Detection Outcome

After execution, Microsoft Sentinel successfully matched the process creation telemetry against the scheduled rule logic and generated a **High-severity alert** tied to **JozefsVM**.

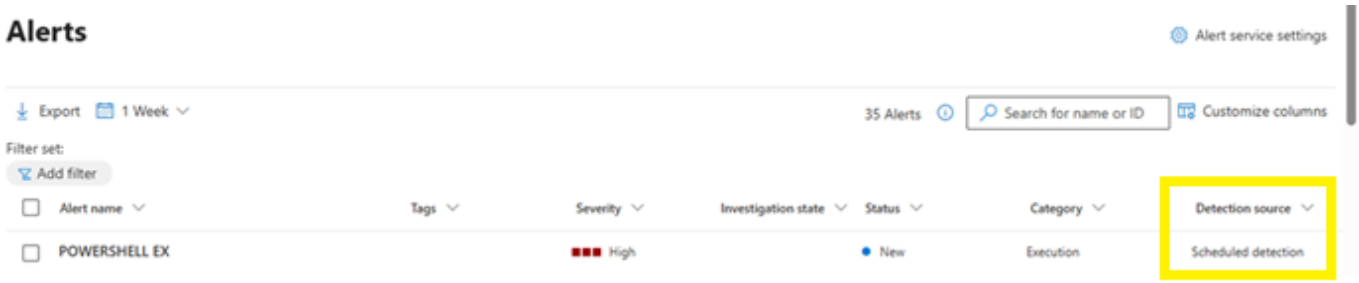
This confirmed:

- endpoint telemetry visibility
- KQL rule logic

- scheduled alert generation
- Defender unified incident correlation

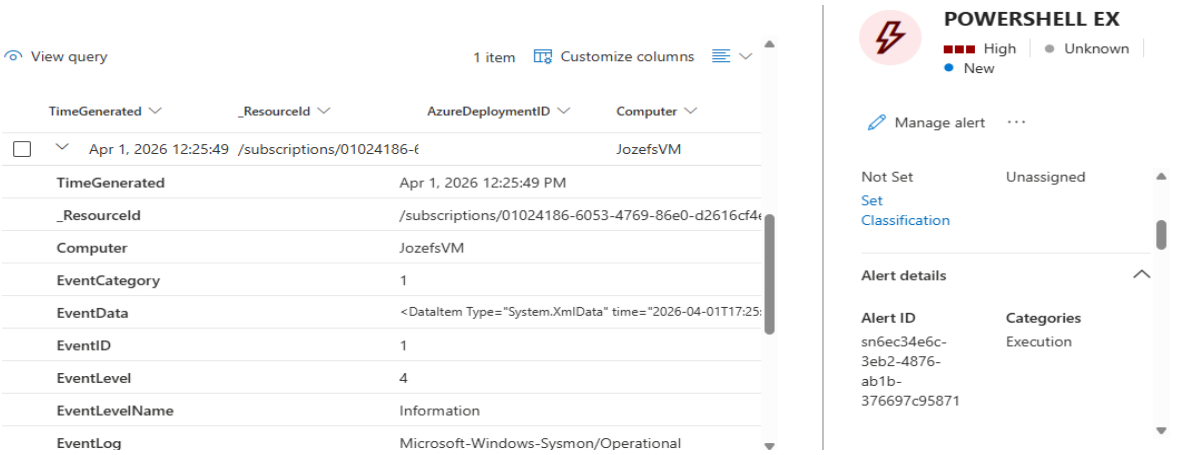
Operational Outcome

The rule successfully generated a **High-severity alert** which was automatically correlated into the **Microsoft Defender unified incident experience**.



The resulting incident included:

- Endpoint: **JozefsVM**
- Sysmon process creation telemetry
- 4 grouped alerts
- Incident severity: **High**
- Incident status: **New**



This validated the full pipeline from:

endpoint telemetry → SIEM detection → scheduled alert → unified SOC incident

This result validated the full production-style detection pipeline, proving that endpoint telemetry was successfully ingested, analyzed by scheduled Sentinel logic, converted into a high-severity alert, and escalated into a unified SOC incident for investigation.

Production Relevance & SOC Value

Encoded PowerShell is frequently used by adversaries for payload staging, in-memory execution, and defense evasion. This detection demonstrates how endpoint process telemetry can be transformed into actionable SOC alerts that reduce attacker dwell time and accelerate analyst triage.